

Three blanks, and a check that tells you the truth

The hardest session of the term, so a skeleton and never a blank page. The forward pass and half of the backward pass are written; you fill three lines, and a gradient check written for you proves them right or wrong.

Guided lab

20 minutes

Three blanks

Gradient check provided

Before you start

Setup · given

1 · one weight

2 · the three blanks

3 · the gradient check ...

4 · one step downhill

Reflection

If you finish early

Before you start

Open the provided `lab10.ipynb`. The gradient check at the end is the same central-difference idea as the `numgrad` you kept from Lab 6, applied to every weight.



By the end of this lab you can

- Complete the backward pass of the 2–3–1 network: three chain-rule lines.
- Read a gradient check and decide whether a derivation is correct.
- Show that one gradient step lowers the loss.

LAB 10

Setup · given

The XOR data, the same 2–3–1 network, small random starting weights, and the forward pass from Lab 9.

given

PYTHON

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

X = np.array([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = np.array([[0.], [1.], [1.], [0.]])

rng = np.random.default_rng(7)
params = {"W1": rng.normal(0, 0.5, (2, 3)), "b1": np.zeros((1, 3)),
          "W2": rng.normal(0, 0.5, (3, 1)), "b2": np.zeros((1, 1))}

def forward(p, X):
    A1 = sigmoid(X @ p["W1"] + p["b1"])
    A2 = sigmoid(A1 @ p["W2"] + p["b2"])
    return A1, A2

def loss(p):
    _, A2 = forward(p, X)
    return -np.mean(y * np.log(A2 + 1e-12) + (1 - y) * np.log(1 - A2 + 1e-12))

print("starting loss:", round(loss(params), 4))
```

what it should print

OUTPUT

starting loss: 0.7002

TASK 1 · 3 MINUTES · ON PAPER

one weight

1

Before the code: write the chain for **one** weight, `W2[0, 0]`, as a product of three factors.

PYTHON

```

$$\frac{\partial L}{\partial W2[0,0]} = \frac{\partial L}{\partial Z2} \times \frac{\partial Z2}{\partial W2[0,0]} = (A2 - y)/m \times A1[:, 0] \quad \text{summed over the 4 examples}$$

```

Then do the same for `W1[0, 0]` — it needs two more factors, because the path goes back through `W2` and through a sigmoid. That longer path is exactly what the three blanks below compute for every weight at once.

the three blanks

Every gradient has **the same shape as the thing it is the gradient of**. Print shapes as you go; a wrong transpose shows up there first.



🔗 The sigmoid's slope, written with its own output, is $A * (1 - A)$.

your turn

PYTHON

```
def backward(p, X, y):
    m = len(X)
    A1, A2 = forward(p, X)
    dZ2 = (A2 - y) / m                # sigmoid + cross-entropy, as in Lab 8
    dW2 = A1.T @ dZ2                  # (3, 1)
    db2 = dZ2.sum(axis=0, keepdims=True) # (1, 1)

    dA1 = None                        # TODO blank 1: carry dZ2 back through W2          → (4, 3)
    dZ1 = None                        # TODO blank 2: through the hidden sigmoid          → (4, 3)
    dW1 = None                        # TODO blank 3: the first layer's weight gradient    → (2, 3)
    db1 = dZ1.sum(axis=0, keepdims=True) # (1, 3)
    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}

grads = backward(params, X, y)
for k in grads:
    print(k, grads[k].shape, "matches" if grads[k].shape == params[k].shape else "SHAPE MISMATCH")
```

one version that works

PYTHON

```
def backward(p, X, y):
    m = len(X)
    A1, A2 = forward(p, X)
    dZ2 = (A2 - y) / m
    dW2 = A1.T @ dZ2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dA1 = dZ2 @ p["W2"].T
    dZ1 = dA1 * A1 * (1 - A1)
    dW1 = X.T @ dZ1
    db1 = dZ1.sum(axis=0, keepdims=True)
    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}

grads = backward(params, X, y)
for k in grads:
    print(k, grads[k].shape, "matches" if grads[k].shape == params[k].shape else "SHAPE MISMATCH")
```

what it should print

OUTPUT

```
W1 (2, 3) matches
b1 (1, 3) matches
W2 (3, 1) matches
b2 (1, 1) matches
```

TASK 3 · 3 MINUTES

the gradient check · given

3

For every single weight and bias: nudge it up by `h`, nudge it down by `-h`, and compare the change in loss with your gradient. It is Lab 6's `numgrad`, run thirteen times.

given

PYTHON

```
def gradient_check(p, grads, h=1e-5):
    worst = 0.0
    for k in p:
        for idx in np.ndindex(p[k].shape):
            old = p[k][idx]
            p[k][idx] = old + h; up = loss(p)
            p[k][idx] = old - h; down = loss(p)
            p[k][idx] = old
            numeric = (up - down) / (2 * h)
            worst = max(worst, abs(numeric - grads[k][idx]))
    return worst

worst = gradient_check(params, grads)
print("largest disagreement:", f"{worst:.2e}", "→", "PASS" if worst < 1e-7 else "FAIL – one
of your blanks is wrong")
assert worst < 1e-7
```

what it should print

OUTPUT

largest disagreement: 8.32e-12 → PASS

TASK 4 • 3 MINUTES

one step downhill

4

The payoff: use the gradients to move every parameter a little, exactly as in Lab 6, and confirm the loss went **down**.

given

PYTHON

```
before = loss(params)
lr = 1.0
for k in params:
    params[k] -= lr * grads[k]
after = loss(params)

print("loss before:", round(before, 5), " after one step:", round(after, 5))
assert after < before
```

what it should print

OUTPUT

loss before: 0.70015 after one step: 0.6955

Reflection



Write it in the notebook, in your own words

Which part of backpropagation is **mathematics**, and which part is **bookkeeping**? Give one concrete line from `backward` as an example of each.

Your answer:

If you finish early

A. Break it on purpose. Delete `* A1 * (1 - A1)` from blank 2 and rerun the check. How large is the disagreement now? A gradient check that can fail is the only kind worth trusting.

B. Midterm review. Without notes, explain in one or two sentences each: *feature, label, parameter, prediction, loss, gradient, learning rate, activation, forward pass, backpropagation*. Then draw the 2–3–1 network and write the shape of every array.

your turn

PYTHON

```
# TODO (optional extension A)
```